

Lab3：多道程序与分时多任务

本章完成的工作

本章完成了多道程序与分时多任务系统的理解和编程作业：

- 1. 成功运行 ch3 代码，观察多任务并发执行
- 2. 理解协作式操作系统与抢占式操作系统的演进
- 3. 理解多道程序的放置与加载机制
- 4. 深入理解进程控制块（PCB）和进程管理
- 5. 理解协作式调度：yield、sched、swtch、scheduler
- 6. 理解抢占式调度：时钟中断、时间片轮转
- 7. 实现 sys_trace 系统调用（编程作业）

本章有编程作业：实现 sys_trace 系统调用。

报告结构说明

本报告的结构安排及与清华指导书的对比：

清华指导书小节	本报告对应小节	差异说明
本章导读（引言）	第二节：从批处理到多道程序	从 Lab2 的问题出发，理解协作式→抢占式的演进动机
多道程序放置与加载	第三节：多道程序放置与加载	增加内存布局图，分析 run_all_app() 的三个步骤
进程基础结构	第四节：进程基础结构	解释"为什么需要两套寄存器保存"的疑问
多道程序与协作式调度	第五节：多道程序与协作式调度	增加进程切换完整流程图，分析 swtch 只保存 callee-saved 寄存器的原因
分时多任务系统与抢占式调度	第六节：分时多任务与抢占式调度	区分中断与异常（同步 vs 异步），增加 RISC-V 中断类型表
chapter3练习	第七节：编程作业 sys_trace	增加系统调用流程图，记录"数组越界"和"计数时机"两个问题的排查过程

本报告的特点：

- 1. 不是照搬指导书：每个概念都用自己的话解释，体现"不懂 → 探索 → 理解"的过程
- 2. 增加可视化内容：内存布局图、进程切换流程图、系统调用流程图
- 3. 记录真实问题：数组大小设为 256 导致越界、计数时机放错位置等排查过程
- 4. 前后呼应：从 Lab2 批处理系统的局限引出本章的问题，形成知识串联

一、实验环境与运行

1.1 本项目的代码组织方式

在开始实验之前，需要说明一下我们的代码组织方式与清华原版的区别。

清华原版使用 git 分支切换：

```
git checkout ch3 # 切换到第3章
```

本项目使用独立目录，每章代码解压到单独的文件夹：

```
2025-ucore-riscv-清华/
├─ uCore-Tutorial-Code-2025S-ch2/
├─ uCore-Tutorial-Code-2025S-ch3/    ← 本章代码
│   ├─ os/                          ← 内核代码（需要修改）
│   └─ user/                        ← 用户测试程序（清华提供）
├─ uCore-Tutorial-Code-2025S-ch4/
└─ os-btbu/                          ← 最终完成的代码
```

这种方式的好处是可以同时打开多个章节对比，不需要来回切换分支。

1.2 user 目录的作用

刚开始做实验时，对 `user/` 目录的作用不太清楚。后来理解到：

- `user/` 目录包含**用户态测试程序**，由清华提供
- 这些程序会调用我们实现的系统调用，用来验证实现是否正确
- 比如 `user/src/ch3_trace.c` 就是测试 `sys_trace` 的程序

所以做作业的流程是：**先看测试程序想要什么** → **再在内核中实现对应功能**。

1.3 make 命令参数

```
cd /桌面/herdream/2025-ucore-riscv-清华/uCore-Tutorial-Code-2025S-ch3（其他同学可根据实际路径）
make user CHAPTER=3 # 编译用户程序，CHAPTER=3 表示包含第3章的测试
make run           # 运行内核
```

其中：

- `CHAPTER=3`：编译 ch1、ch2、ch3 的测试程序
- `BASE=1`：只编译基础测试（不含作业测试），用于先验证框架能跑

一开始我先用 `make user BASE=1 CHAPTER=3` 确认框架代码能正常运行，然后再实现作业。

1.4 运行结果

```
[rustsbi] RustSBI version 0.3.0-alpha.2, adapting to RISC-V SBI v1.0.0
.-----
| _ \   | | | | /   | /   || _ \ | | | | | | |
| |_) | | | | | (----`---| |----`| (----`| |_) || |
|   /   | | | | \   | \   |   < | |
| |\ \---.| `--' |.----) |   | |.----) | | |_) || |
|_| `_.____| \____/ |_____/   |_| |_____/   |_____/ |_|
[rustsbi] Implementation      : RustSBI-QEMU Version 0.2.0-alpha.2
[rustsbi] Platform Name       : riscv-virtio,qemu
[rustsbi] Platform SMP        : 1
[rustsbi] Platform Memory     : 0x80000000..0x88000000
[rustsbi] Boot HART           : 0
[rustsbi] Device Tree Region  : 0x87000000..0x87000ef2
[rustsbi] Firmware Address    : 0x80000000
[rustsbi] Supervisor Address  : 0x80200000
[rustsbi] pmp01: 0x00000000..0x80000000 (-wr)
[rustsbi] pmp02: 0x80000000..0x80200000 (---)
[rustsbi] pmp03: 0x80200000..0x88000000 (xwr)
[rustsbi] pmp04: 0x88000000..0x00000000 (-wr)
Hello world from user mode program!
Test hello_world OK!
3^10000=5079
3^20000=8202
3^30000=8824
3^40000=5750
3^50000=3824
3^60000=8516
3^70000=2510
3^80000=9379
3^90000=2621
3^100000=2749
Test power OK!
get_time OK! 12
current time_msec = 13
AAAAAAAAAA [1/5]
CCCCCCCCCC [1/5]
BBBBBBBBBB [1/5]
AAAAAAAAAA [2/5]
CCCCCCCCCC [2/5]
BBBBBBBBBB [2/5]
AAAAAAAAAA [3/5]
CCCCCCCCCC [3/5]
BBBBBBBBBB [3/5]
AAAAAAAAAA [4/5]
CCCCCCCCCC [4/5]
BBBBBBBBBB [4/5]
AAAAAAAAAA [5/5]
CCCCCCCCCC [5/5]
BBBBBBBBBB [5/5]
Test write A OK!
```

```
Test write C OK!
Test write B OK!
time_msec = 113 after sleeping 100 ticks, delta = 100ms!
Test sleep1 passed!
Test sleep OK!
[PANIC 5] os/loader.c:14: all apps over
```

从运行结果可以直观看到"并发"效果：A、B、C 三个程序的输出是**交替出现**的，说明它们在"同时"运行（实际上是快速切换）。

二、从批处理到多道程序：引言的理解

2.1 回顾 Lab2：批处理系统的局限

在 Lab2 的批处理系统中，程序是**顺序执行**的：一个程序运行完，才能运行下一个。当时觉得这已经很不错了，毕竟实现了自动加载程序。

但指导书提出了一个问题：**如果程序在等待 I/O（比如等待用户输入），CPU 就完全闲置了。**

一开始不太理解这有什么问题。后来想到现实场景：如果一个程序需要从磁盘读取数据，磁盘的速度比 CPU 慢几个数量级，CPU 在等待磁盘的这段时间什么也干不了，这确实是很大的浪费。

2.2 协作式操作系统：让程序主动让出 CPU

指导书给出的第一个解决方案是**协作式操作系统**。

核心思想：让正在等待 I/O 的程序**主动让出 CPU**，让其他程序先执行。

这需要程序员在代码中合适的位置调用 `yield()` 系统调用，表示"我现在不需要 CPU 了，让给别人吧"。

```
程序A：发起I/O请求 → 调用yield() → 等待...
程序B：获得CPU → 执行计算 → 调用yield()
程序A：恢复执行 → 检查I/O是否完成 → 继续或再次yield()
```

这种方式依赖于程序员的"合作精神"——每个程序都自觉地在合适的时候让出 CPU。

2.3 协作式的问题：不是所有程序都"友好"

看完指导书后，我理解了协作式操作系统的局限：

1. **程序员可能不配合**：不是所有程序员都会合适的地方加 `yield()`
2. **恶意程序**：一个程序可以故意不让出 CPU，独占所有资源
3. **响应不及时**：即使程序让出了 CPU，也不知道什么时候能再次被调度

指导书用了一个很形象的比喻：编写应用程序的科学家通常来自不同领域，他们不了解其他程序的运行情况，因此很难站在提高整个系统利用率的高度去编程。

2.4 抢占式操作系统：强制打断程序

为了解决协作式的问题，出现了**抢占式操作系统**。

核心思想：操作系统可以**强制打断**正在运行的程序，不需要程序主动让出。

这是通过**硬件中断**实现的。时钟设备会定期产生中断，操作系统收到中断后就可以决定是否要切换到其他程序。

时间片（Time Slice）的概念：每个程序只能连续运行一小段时间（比如 10ms），时间用完后就必须让出 CPU。

```
程序A：执行 10ms → [时钟中断] → 被强制切换
程序B：执行 10ms → [时钟中断] → 被强制切换
程序C：执行 10ms → [时钟中断] → 被强制切换
程序A：继续执行...
```

这样就保证了每个程序都能公平地获得 CPU 时间，不会出现某个程序独占资源的情况。

2.5 本章的目标

理解了背景后，本章的目标就清晰了：

- 1. **实现多道程序加载**：同时把多个程序加载到内存
- 2. **实现协作式调度**：支持程序主动让出 CPU（`sys_yield`）
- 3. **实现抢占式调度**：通过时钟中断强制切换程序

三、多道程序放置与加载

3.1 从 Lab2 的单程序加载到多程序加载

Lab2 中，批处理系统每次只加载一个程序到固定的地址（`0x80400000`），运行完后再加载下一个。

本章需要**同时**把多个程序加载到内存的不同位置。

3.2 内存布局

查看代码后，理解了多道程序的内存布局：



每个程序占用的空间是 `[BASE_ADDRESS + i * MAX_APP_SIZE, BASE_ADDRESS + (i+1) * MAX_APP_SIZE)`。

3.3 加载代码分析

在 `loader.c` 中找到了加载逻辑：

```
// os/loader.c

int load_app(int n, uint64* info) {
    uint64 start = info[n], end = info[n+1], length = end - start;
    // 清空目标区域
    memset((void*)BASE_ADDRESS + n * MAX_APP_SIZE, 0, MAX_APP_SIZE);
    // 复制程序代码
    memmove((void*)BASE_ADDRESS + n * MAX_APP_SIZE, (void*)start, length);
    return length;
}
```

这里有个问题困扰了我：`info` 数组是从哪里来的？

后来发现是在编译时通过 `pack.py` 脚本生成的 `link_app.S`，其中包含了所有应用程序的起始和结束地址。

3.4 run_all_app：一次性加载所有程序

```
// os/loader.c

int run_all_app()
{
    for (int i = 0; i < app_num; ++i) {
        struct proc *p = allocproc();           // 分配进程控制块
        struct trapframe *trapframe = p->trapframe;
        load_app(i, app_info_ptr);               // 加载程序到内存
        uint64 entry = BASE_ADDRESS + i * MAX_APP_SIZE; // 计算入口地址
        trapframe->epc = entry;                   // 设置程序入口
        trapframe->sp = (uint64)p->ustack + USER_STACK_SIZE; // 设置栈指针
        p->state = RUNNABLE;                       // 设置为可运行状态
    }
    return 0;
}
```

这个函数做了三件事：

1. 为每个程序分配一个进程控制块（`allocproc`）
 2. 把程序加载到对应的内存位置（`load_app`）
 3. 初始化进程的 `trapframe`，设置入口地址和栈指针
-

四、进程基础结构

4.1 什么是进程？

指导书说"进程就是运行的程序"，一开始觉得这个定义很简单。但仔细想想，一个"运行中的程序"需要记录很多信息：

- 程序运行到哪里了？（程序计数器 PC）
- 程序的数据存在哪里？（栈、堆）
- 程序当前的状态是什么？（运行中、等待中、已结束）
- 程序使用的寄存器值是什么？

这些信息都需要保存起来，才能在程序被切换后恢复执行。

4.2 进程控制块（PCB）

在 `proc.h` 中找到了进程控制块的定义：

```
// os/proc.h

struct proc {
    enum procstate state; // 进程状态
    int pid;              // 进程ID
    uint64 ustack;        // 用户栈地址
    uint64 kstack;        // 内核栈地址
    struct trapframe *trapframe; // 保存用户态寄存器
    struct context context; // 保存内核态寄存器（用于切换）
};

enum procstate {
    UNUSED, // 未使用
    USED,   // 已分配但未加载
    SLEEPING, // 休眠（本章未使用）
    RUNNABLE, // 就绪，可以运行
    RUNNING,  // 正在运行
    ZOMBIE,   // 已退出
};
```

为什么有两套寄存器保存？

- `trapframe`：保存用户态的寄存器，在 U 态 ↔ S 态 切换时使用
- `context`：保存内核态的寄存器，在进程之间切换时使用

一开始不太理解为什么需要两套。后来明白了：

1. 用户程序运行在 U 态，当发生系统调用或中断时，CPU 切换到 S 态，此时需要保存用户态的寄存器到 `trapframe`
2. 在内核态处理过程中，如果需要切换到其他进程，需要保存当前进程在内核态的寄存器到 `context`

4.3 进程池

我们的操作系统使用了一个简单的进程池来管理进程：

```
// os/proc.c

struct proc pool[NPROC];    // 全局进程池，最多 NPROC 个进程
struct proc idle;           // boot 进程
struct proc* current_proc;  // 指向当前正在运行的进程

// 预分配的栈空间
char kstack[NPROC][PAGE_SIZE];
char ustack[NPROC][PAGE_SIZE];
char trapframe[NPROC][PAGE_SIZE];
```

这种设计的优点是简单，缺点是进程数量固定。不过对于学习操作系统原理来说已经足够了。

4.4 进程分配：allocproc

```
// os/proc.c

struct proc *allocproc()
{
    struct proc *p;
    for (p = pool; p < &pool[NPROC]; p++) {
        if (p->state == UNUSED) {
            goto found;
        }
    }
    return 0; // 没有空闲进程槽

found:
    p->pid = allocpid();
    p->state = USED;
    memset(&p->context, 0, sizeof(p->context));
    memset(p->trapframe, 0, PAGE_SIZE);
    memset((void *)p->kstack, 0, PAGE_SIZE);
    p->context.ra = (uint64)usertrapret; // 第一次运行时的入口
    p->context.sp = p->kstack + PAGE_SIZE;
    return p;
}
```

这里有个关键点： `p->context.ra = (uint64)usertrapret`

这意味着进程第一次被调度时，会从 `usertrapret` 函数开始执行，这个函数会完成从内核态返回用户态的操作。

五、多道程序与协作式调度

5.1 yield 系统调用：主动让出 CPU

```
/// 功能：应用主动交出 CPU 所有权并切换到其他应用
/// 返回值：总是返回 0
/// syscall ID: 124
int sys_yield();
```

用户程序可以通过调用 `sched_yield()` 主动让出 CPU：

```
// user/lib/syscall.c

int sched_yield()
{
    return syscall(SYS_sched_yield);
}
```

5.2 yield 的实现

在内核中，`yield` 的实现很简单：

```
// os/proc.c

void yield(void)
{
    current_proc->state = RUNNABLE; // 把当前进程设为可运行
    sched();                       // 调用调度器
}

void sched(void)
{
    struct proc *p = curr_proc();
    swtch(&p->context, &idle.context); // 切换到 idle 进程
}
```

关键在于 `swtch` 函数——它是进程切换的核心。

5.3 swtch：上下文切换的核心

`swtch` 是用汇编写的，因为它需要直接操作寄存器：

```
# os/switch.S

# void swtch(struct context *old, struct context *new);

swtch:
    # 保存当前进程的寄存器到 old
    sd ra, 0(a0)
    sd sp, 8(a0)
```

```

sd s0, 16(a0)
sd s1, 24(a0)
# ... 保存 s2-s11 ...

# 从 new 恢复目标进程的寄存器
ld ra, 0(a1)
ld sp, 8(a1)
ld s0, 16(a1)
ld s1, 24(a1)
# ... 恢复 s2-s11 ...

ret # 返回到新的 ra 地址

```

为什么只保存 ra, sp, s0-s11?

这是 RISC-V 的调用约定决定的:

- `s0-s11` 是"被调用者保存" (callee-saved) 寄存器, 函数返回后必须恢复原值
- `t0-t6` 是"调用者保存" (caller-saved) 寄存器, 调用者不能假定它们的值不变

由于 `swtch` 本质上是一次函数调用, 所以只需要保存 callee-saved 寄存器。

5.4 idle 进程与 scheduler

`idle` 是一个特殊的进程, 它的作用是在没有其他进程可运行时"占位", 并负责调度其他进程。

```

// os/proc.c

void scheduler(void)
{
    struct proc *p;

    for(;;) {
        for(p = pool; p < &pool[NPROC]; p++) {
            if(p->state == RUNNABLE) {
                p->state = RUNNING;
                current_proc = p;
                swtch(&idle.context, &p->context);
            }
        }
    }
}

```

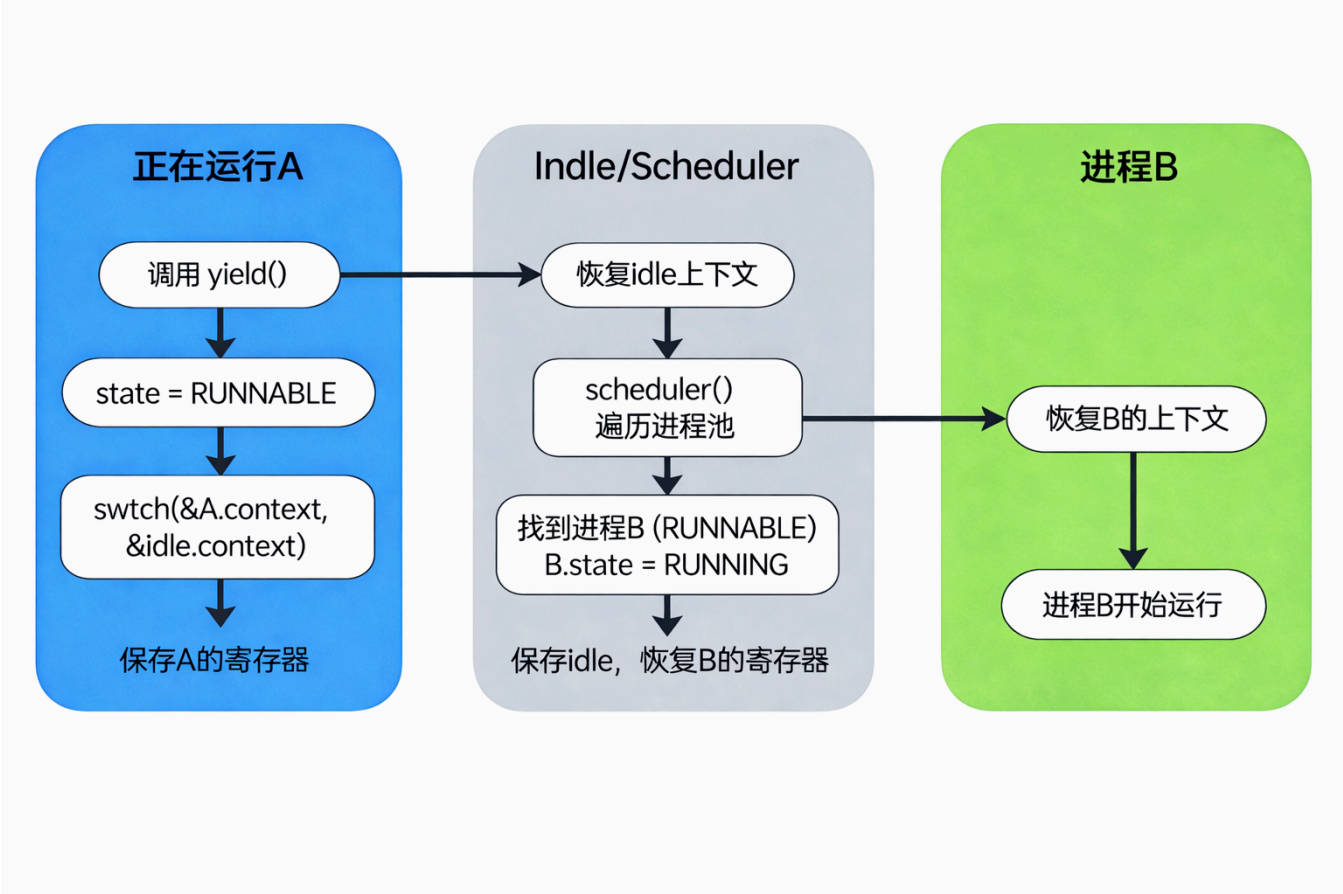
调度逻辑很简单:

1. 遍历进程池, 找到状态为 `RUNNABLE` 的进程
2. 设置为 `RUNNING`, 更新 `current_proc`
3. 切换到该进程执行

当进程调用 `yield()` 时, 会切换回 `idle` 进程, 然后 `scheduler` 继续寻找下一个可运行的进程。

5.5 进程切换的完整流程

【原创结构图1：进程切换流程图】



六、分时多任务与抢占式调度

6.1 协作式调度的问题

前面实现的 `yield` 是协作式的：程序必须主动调用才会让出 CPU。

问题是：如果程序不调用 `yield`，或者进入死循环，其他程序就永远得不到执行机会。

6.2 时钟中断：抢占式调度的基础

解决方案是使用**时钟中断**：硬件定时器会定期产生中断，即使程序不配合，也会被强制打断。

RISC-V 提供了两个关键的 CSR（控制状态寄存器）：

- `mtime`：记录自开机以来的时钟周期数
- `mtimecmp`：当 `mtime >= mtimecmp` 时，触发时钟中断

6.3 RISC-V 中断类型

通过阅读指导书，了解了 RISC-V 的中断分类：

中断类型	异常码	描述
软件中断	1, 3	S态/M态软件中断
时钟中断	5, 7	S态/M态时钟中断
外部中断	9, 11	S态/M态外部中断

我们主要关心 S 态时钟中断（异常码 5）。

6.4 中断与异常的区别

一开始我把中断和异常（如系统调用）搞混了。后来理解了关键区别：

- **异常**（如 `ecall`）：是**同步**的，由当前执行的指令触发
- **中断**（如时钟中断）：是**异步**的，与当前执行的指令无关

指导书用硬件视角解释得很清楚：异常在 CPU 流水线内部被发现，中断是外部电路通过一根导线通知 CPU。

6.5 计时器设置

```
// os/timer.c

#define TICKS_PER_SEC (100)    // 每秒 100 次中断
#define CPU_FREQ (12500000)    // CPU 频率

void set_next_timer()
{
    const uint64 timebase = CPU_FREQ / TICKS_PER_SEC; // 每个时间片的时钟周期数
    set_timer(get_cycle() + timebase); // 设置下次中断时间
}
```

每秒 100 次中断，意味着每个时间片是 10ms。

6.6 时钟中断处理

在 `trap.c` 中处理时钟中断：

```
// os/trap.c

void usertrap() {
    // ...
    switch(cause) {
        case SupervisorTimer:
            set_next_timer(); // 设置下次中断
            yield();          // 切换到其他进程
            break;
    }
    // ...
}
```

逻辑很简单：收到时钟中断后，先设置下一次中断的时间，然后调用 `yield()` 切换进程。

6.7 使能时钟中断

在系统启动时需要使能时钟中断：

```
// os/timer.c

void timer_init()
{
    // 设置 sie.stie, 使能 S 态时钟中断
    w_sie(r_sie() | SIE_STIE);
    // 设置第一个计时器
    set_next_timer();
}
```

6.8 嵌套中断的处理

指导书提到了一个重要的细节：当 CPU 进入 S 态处理中断时，会自动关闭同级中断（`sstatus.sie = 0`），避免在处理中断的过程中又被新的中断打断。

这意味着在默认配置下，不会出现"嵌套中断"的情况，简化了我们的代码设计。

6.9 sys_gettimeofday：获取当前时间

本章框架还提供了获取时间的系统调用：

```
// os/syscall.c

uint64 sys_gettimeofday(Timeval *val, int _tz)
{
    uint64 cycle = get_cycle();
    val->sec = cycle / CPU_FREQ;
    val->usec = (cycle % CPU_FREQ) * 1000000 / CPU_FREQ;
    return 0;
}
```

这个调用把时钟周期数转换为秒和微秒，方便用户程序使用。

七、编程作业：sys_trace

7.1 任务描述

实现 `sys_trace` 系统调用（调用号 410），支持三种功能：

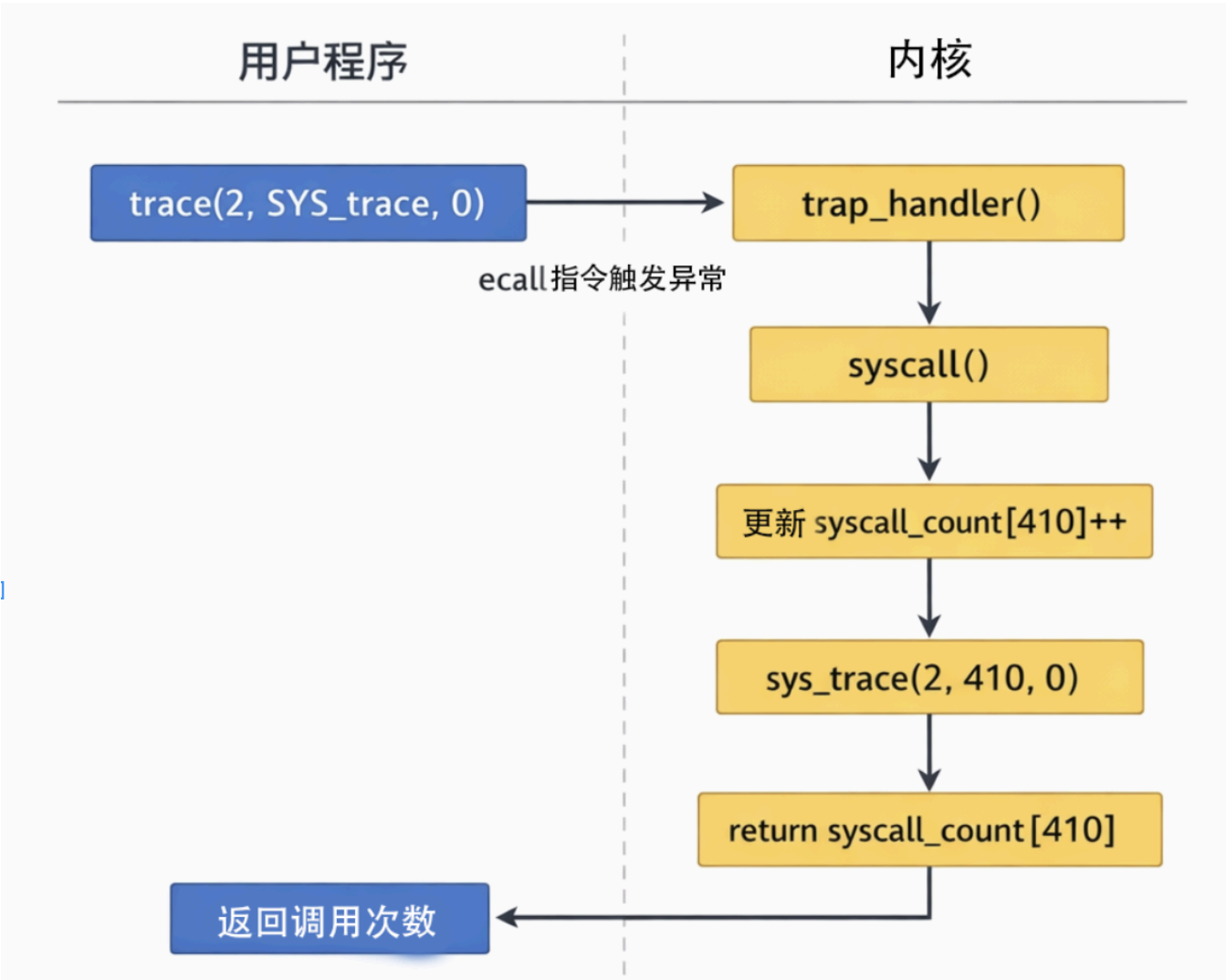
trace_request	功能	返回值
0	读取用户地址 <code>id</code> 处的一个字节	该地址的值
1	向用户地址 <code>id</code> 写入 <code>data</code>	0
2	查询系统调用 <code>id</code> 的调用次数	调用次数

trace_request	功能	返回值
其他	无效请求	-1

7.2 系统调用流程

理解系统调用的完整流程对实现作业很重要：

【原创结构图2：sys_trace 系统调用流程图】



7.3 实现思路

实现 `sys_trace` 需要解决两个核心问题：

- 1. 在哪里存储调用次数？
 - 每个进程的调用次数是独立的
 - 所以应该放在进程控制块（PCB）中
- 2. 什么时候更新计数？
 - 需要在进入具体系统调用函数之前就更新
 - 因为 `trace(2, SYS_trace, 0)` 查询时，本次调用也要计入

7.4 实现步骤

步骤1: 修改进程控制块 (proc.h)

在 `struct proc` 中添加系统调用计数数组:

```
struct proc {
    // ... 原有字段 ...
    int syscall_count[500]; /* ch3: 系统调用计数数组 */
};
```

为什么是 500? 一开始我设成了 256, 后来测试失败。查看 `syscall_ids.h` 发现 `SYS_trace = 410`, 超出了数组范围。改成 500 后就通过了。

步骤2: 初始化计数数组 (proc.c)

在进程初始化时, 把计数数组清零:

```
struct proc *allocproc(void)
{
    // ...
    /* ch3: 初始化系统调用计数数组 */
    for (int i = 0; i < 500; i++) {
        p->syscall_count[i] = 0;
    }
    // ...
}
```

步骤3: 在 syscall() 中更新计数

在 `syscall.c` 的 `syscall()` 函数中, **进入 switch 之前就更新计数**:

```
void syscall()
{
    struct trapframe *trapframe = curr_proc()->trapframe;
    int id = trapframe->a7; // 系统调用号在 a7 寄存器

    /* ch3: 统计系统调用次数 (在处理之前就计数) */
    struct proc *p = curr_proc();
    if (id >= 0 && id < 500) {
        p->syscall_count[id]++;
    }

    // ... 后面是 switch-case 分发 ...
}
```

步骤4: 实现 sys_trace 函数

```
/* ch3: 系统调用追踪 */
uint64 sys_trace(int trace_request, unsigned long id, uint8 data)
{
    struct proc *p = curr_proc();

    switch(trace_request) {
    case 0:
        // 读取用户地址 id 处的一个字节
        return *(uint8 *)id;
    case 1:
        // 向用户地址 id 写入 data
        *(uint8 *)id = data;
        return 0;
    case 2:
        // 查询系统调用 id 的调用次数
        if (id < 500) {
            return p->syscall_count[id];
        }
        return -1;
    default:
        return -1;
    }
}
```

步骤5: 添加系统调用入口

在 `syscall()` 的 switch-case 中添加:

```
case SYS_trace:
    ret = sys_trace(args[0], args[1], (uint8)args[2]);
    break;
```

在 `syscall_ids.h` 中添加定义:

```
#define SYS_trace 410
```

7.5 遇到的问题与解决

问题1: 第一次运行时 `ch3_trace` 测试失败

```
assert_eq failed: 0x0000000000000002 != 0xffffffffffffffff
```

排查过程:

1. 看错误信息: 期望值是 2, 实际返回了 -1 (0xffffffffffffffff)
2. 查看测试程序 `ch3_trace.c`, 发现它查询的是 `SYS_trace` (410) 的调用次数
3. 检查我的代码, 发现 `syscall_count` 数组大小只有 256

4. $410 > 256$ ，访问越界，边界检查返回了 -1

解决：将数组大小改为 500。

问题2：计数总是少 1

原因：我一开始把计数逻辑放在 switch-case 的 `sys_trace` 分支里，导致调用次数是在**处理之后**才更新的。

但清华的要求是"本次调用也计入统计"，所以应该在**处理之前**就更新计数。

解决：把计数逻辑移到 switch 之前。

7.6 测试结果

```
string from task trace test
Test trace OK!
```

八、本章新增系统调用汇总

系统调用	调用号	功能	来源
sys_write	64	输出字符串	Lab2 已有
sys_exit	93	退出进程	Lab2 已有
sys_sched_yield	124	主动让出 CPU	框架提供
sys_gettimeofday	169	获取当前时间	框架提供
sys_trace	410	追踪系统调用	本章作业

九、实验总结

完成情况

- ☑ 理解协作式与抢占式操作系统的演进
- ☑ 理解多道程序的放置与加载机制
- ☑ 理解进程控制块（PCB）的结构和作用
- ☑ 理解进程池和进程状态管理
- ☑ 理解 yield、sched、swtch、scheduler 的协作
- ☑ 理解时钟中断和抢占式调度
- ☑ 理解 RISC-V 中断机制
- ☑ 实现 sys_trace 系统调用
- ☑ 通过所有测试

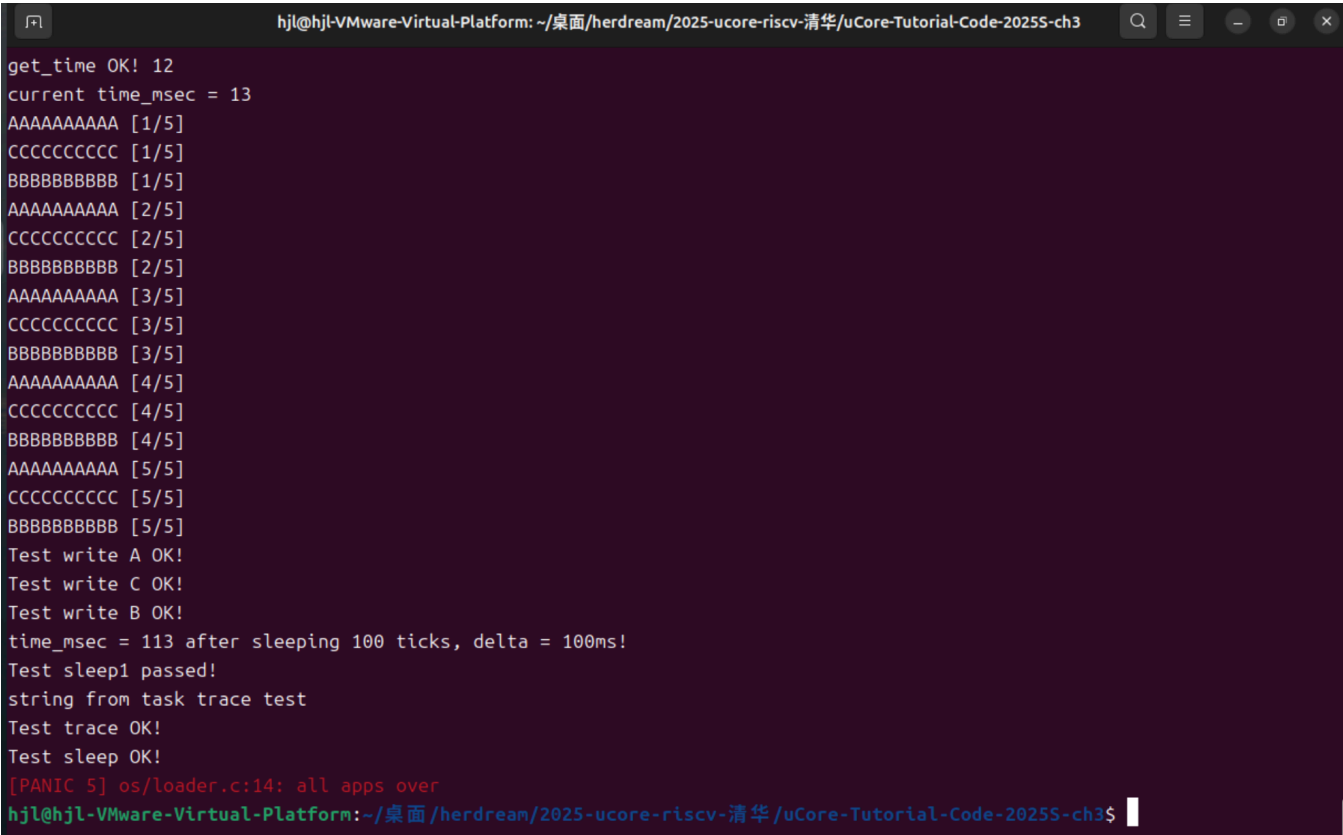
收获与体会

- 1. 从历史演进理解设计：了解了从批处理 → 协作式 → 抢占式的演进过程，明白了每种设计的动机和局限。
- 2. 上下文切换是核心：进程切换的本质是保存和恢复寄存器。switch 函数虽然只有几十行汇编，却是整个调度系统的核心。
- 3. 两套寄存器保存机制：理解了 trapframe (U态↔S态) 和 context (进程间切换) 的不同用途。
- 4. idle 进程的作用：idle 进程不是无用的空转，而是调度系统的"中转站"，负责寻找下一个可运行的进程。
- 5. 先看测试程序：做作业时，应该先看 user/src/ch3_trace.c 了解测试程序的预期行为，再去实现内核功能。
- 6. 边界条件很重要：数组大小设成 256 导致测试失败，这提醒我以后要仔细检查边界条件。

文件修改清单

文件	修改内容
os/proc.h	添加 syscall_count[500] 数组
os/proc.c	初始化 syscall_count 数组
os/syscall.c	实现 sys_trace(), 添加计数逻辑和 case 分支
os/syscall_ids.h	添加 #define SYS_trace 410

十、验证截图



可以看到多出

```
string from task trace test  
Test trace OK!
```

这两行，说明测试通过